Ahsanullah University of Science & Technology
Department of Computer Science & Engineering

A Truck

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By:	
Ananto Nayan Bala	20210204028
Tasmia Hossain	20210204038

1. Problem Analysis

1.1 Project Requirements

The primary goal of this project is to render a 3D scene of a heavy freight truck on a road using Three.js. The scene must incorporate the following core requirements:

- Two distinct 3D objects: a Truck Body and Truck Wheels, each with unique textures.
- Custom GLSL shaders: a shared vertex shader and two separate fragment shaders for the body and wheels.
- Implementation of multiple light sources: HemisphereLight, DirectionalLight, and PointLight.
- Perspective projection camera with orbit, zoom, and auto-follow behavior.
- Procedural canvas-based textures for the truck body, wheels, and road — no external image files required.
- Continuous wheel rotation animation and an orbiting inspection light animation.
- Keyboard interaction to drive, steer, brake, and control camera and light settings.
- Mouse interaction to orbit the camera by dragging and zoom using the scroll wheel.

1.2 Core Integration

The truck was constructed entirely from Three.js primitives — BoxGeometry for the cabin and cargo body, CylinderGeometry for wheels and exhaust pipes, and PlaneGeometry for windows. All truck parts are grouped under a single THREE.Group so that driving and steering transformations apply uniformly.

Two custom ShaderMaterials are used: one for the truck body (bodyMaterial) and one for the wheels (wheelMaterial). Both share a common vertex shader that computes view-space position and normal. The body fragment shader implements Blinn-Phong diffuse and specular lighting plus a rim glow. The wheel fragment shader adds a procedural tread pattern using a sin() function on the UV coordinates.

Textures are generated at runtime using HTML5 Canvas. The body texture includes painted stripes, panel lines, and a company label. The wheel texture features alternating tread columns, spoke shapes, and a hub circle. The road texture includes lane markings. No external image files are needed, making the project fully self-contained.

The camera uses a spherical coordinate system (theta, phi, radius) updated every frame. It auto-orbits the truck when autoCamera is on, and responds to mouse drag and scroll for manual control. The PointLight orbits the truck on a circular path of radius 9 units, while a glowing sphere (lightOrb) and a torus ring (lightTrail) visually indicate its position..

1.3 Technical Challenges

Shader Uniform Synchronization: The custom ShaderMaterial requires the light position to be provided in view space (camera space), not world space. Each frame, the world-space PointLight position is transformed using camera.matrixWorldInverse before being passed to the uLightPosView uniform:

```
tempLightPos.copy(pointLight.position).applyMatrix4(camera.matrixWorldInverse);
```

Wheel Rotation: Wheel spin is computed from the truck speed and wheel radius to achieve physically consistent rolling. The rotation per frame is:

```
wheelSpin = state.speed * dt / wheelRadius;
spinNode.rotation.z -= wheelSpin;
```

Front Wheel Steering: The two front wheel assemblies are placed in separate steeringPivot groups. Their local Y-rotation is set equal to the steer angle, producing realistic Ackermann-like steering visuals.

Road Curve Geometry: The road is generated from a CatmullRomCurve3 spline with custom BufferGeometry built by sampling points along the curve, computing tangent and lateral vectors, and placing vertex pairs to form a ribbon mesh. This produces a naturally curved road without relying on standard primitives.

2. User Interaction

2.1 Key Interaction

Keyboard events are captured using keydown and keyup listeners that maintain a keys object. Each frame, the animate loop reads the current key state to update the truck and scene:

Key	Action	Implementation
W / Arrow Up	Drive Forward	Adds positive speed via acceleration * dt
S / Arrow Down	Drive Backward	Adds negative speed, truck moves in reverse
A / Arrow Left	Steer Left	steer lerps to -0.48, heading decreases
D / Arrow Right	Steer Right	steer lerps to +0.48, heading increases
Space	Brake	speed *= 0.88 per frame (rapid deceleration)
C	Toggle Auto Camera	Toggles cameraOrbitAngle auto-increment on/off
L	Toggle Rotating Light	Toggles lightOrbitAngle auto-increment on/off
H (hold)	Horn Smoke	Emits smoke particles from the front exhaust pipe
R	Reset Scene	Resets truck position, speed, and all state values

Table 01: Keyboard Interaction Summary

2.2 Mouse Interaction

Mouse interaction is handled through pointer events on the canvas element, which supports both mouse and touch input:

- Drag (pointerdown + pointermove): When the user clicks and drags on the canvas, dx and dy are computed from the pointer delta each frame. These are applied to cameraYawOffset and cameraPitch respectively, rotating the camera around the truck in any direction.
- Scroll Wheel (wheel event): Scrolling adjusts cameraDistance, clamped between 6 and 24 units, providing smooth zoom in and out around the truck.
- Horn Button (pointerdown on button): Pressing the on-screen Horn Smoke button sets hornPointerActive = true, triggering the smoke particle system from the front exhaust pipe.

The camera updates every frame using the spherical coordinates (cameraOrbitAngle + cameraYawOffset, cameraPitch, cameraDistance), always pointing at a target 1.9 units above the truck center.

3. Design and Aesthetic Aspects

3.1 Visual Elements

Truck Body Texture: Generated on a 1024x512 canvas. A warm orange-to-dark-red linear gradient forms the base. Semi-transparent horizontal stripes add a metal panel effect. Two gold accent bands are drawn at the top and bottom. Vertical panel division lines and horizontal cross-bars add structural detail. The company name ROADLINE LOGISTICS and subtitle HEAVY FREIGHT DIVISION are rendered in dark text, simulating a real freight truck livery.

Wheel Texture: Generated on a 768x768 canvas. Alternating dark column stripes simulate the tyre tread surface. Concentric circles mark the sidewall. Ten rectangular spokes radiate from the center. A circular hub cap with a lighter fill completes the wheel face. In the fragment shader, a sin() function on the U coordinate adds further tread modulation dynamically.

Road Texture: Generated on a 1024x1024 canvas. A dark grey base with random noise patches creates an aged asphalt look. Dashed white centre line markings are drawn at regular intervals.

Lighting Model: The truck body and wheels use a custom Blinn-Phong lighting model implemented in GLSL. The body fragment shader computes diffuse, specular (shininess exponent 36), and a rim highlight term. The rim term brightens edges facing away from the camera, giving the truck a stylised metallic appearance. The wheel shader adds an edge glow using the same principle, tinted with a cool blue-white color.

Scene Atmosphere: A dark blue-grey background color and fog (Fog, near 18, far 60) blend distant objects into the environment. A golden PointLight orbits the truck with a visible light orb mesh and a translucent torus ring marking its circular path. This makes the rotating light feature immediately visible and visually interesting.

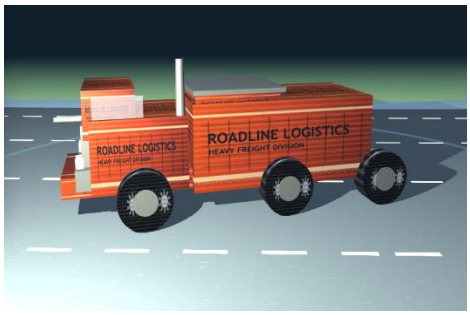
3.2 Snapshots

The following features are visible in the project screenshots:

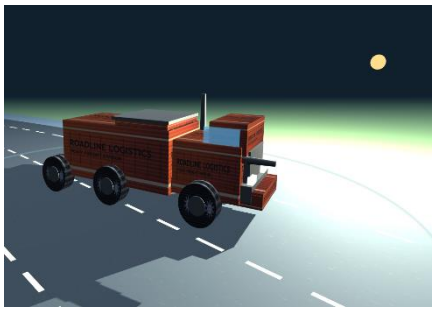
- Front view: Truck headlights, grille, bumper, windshield with glass material, and side mirrors.
- Side view: Cargo body with ROADLINE LOGISTICS texture, exhaust pipe, three axles with dual-tyre appearance, and road with lane markings.
- Light orbit: The golden light orb and teal torus trail show the orbiting PointLight position.

- Smoke effect: When H is held or the Horn Smoke button is pressed, dark smoke particles emit from the front exhaust pipe tip and rise with slight lateral drift.
- Driving: W/S keys accelerate/reverse the truck along the road; A/D keys steer with front wheel pivot animation.

Front View

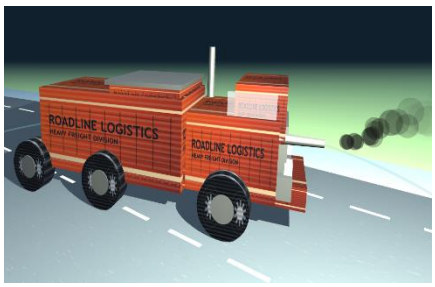


Side View


Orbiting Light

Wheel Rotation

Steering



Horn Smoke

4. Code Implementation

4.1 Software Platform

The project is implemented as a single self-contained HTML file with no build step required. The following tools and technologies are used:

- HTML5 / CSS3: Page structure, HUD panel styling, and responsive layout.
- JavaScript (ES Modules): All Three.js scene logic, interaction handling, and animation loop.
- Three.js (r161): 3D rendering library loaded from the local js/ folder via importmap.
- WebGL: Underlying GPU rendering API used by Three.js.
- GLSL (OpenGL Shading Language): Custom vertex and fragment shaders embedded as template literal strings.
- HTML5 Canvas API: Procedural texture generation for truck body, wheels, and road at runtime.
- VS Code + Live Server: Development environment and local HTTP server for running the project.
- Node.js: Required as a prerequisite for the Three.js local module setup.

4.2 Custom Shaders

Both the truck body and the wheels use THREE.ShaderMaterial with custom GLSL shaders. A single shared vertex shader is used for both materials.

Vertex Shader: The vertex shader transforms each vertex from object space to clip space and passes the interpolated UV coordinates, view-space position, and view-space normal to the fragment shader:

```
vUv = uv;
vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);
vViewPosition = mvPosition.xyz;
vViewNormal = normalize(normalMatrix * normal);
gl_Position = projectionMatrix * mvPosition;
```

Body Fragment Shader: Implements Blinn-Phong shading with a rim highlight. The texture color is sampled and modulated by ambient, diffuse, and specular terms. A rim term brightens silhouette edges facing away from the camera:

```
float diffuse  = max(dot(normal, lightDir), 0.0);
float specular = pow(max(dot(normal, halfVector), 0.0), 36.0);
float rim      = pow(1.0 - max(dot(normal, viewDir), 0.0), 2.3);
vec3 color = base * (uAmbientColor + diffuse * uLightColor);
color += specular * uSpecularColor + rim * 0.07 * uLightColor;
```

Wheel Fragment Shader: Adds a procedural tread modulation using a sin function on the U coordinate, then applies diffuse lighting and an edge glow:

```
float tread = 0.5 + 0.5 * sin(vUv.x * 90.0);
tex *= mix(0.78, 1.03, tread);
float edge = pow(1.0 - max(dot(normal, viewDir), 0.0), 3.0);
color += edge * 0.18 * vec3(0.85, 0.9, 1.0);
```

4.3 Project Feature Table:

#	Feature	Status
1	Truck Body (Cabin + Cargo) with Procedural Canvas Texture	Implemented
2	Truck Wheels with Custom Tyre & Hub Texture	Implemented
3	Custom Vertex Shader (shared for body and wheels)	Implemented
4	Custom Fragment Shader — Body (Blinn-Phong diffuse + specular + rim)	Implemented
5	Custom Fragment Shader — Wheel (sin-based procedural tread pattern)	Implemented
6	Perspective Projection Camera (PerspectiveCamera, FOV 52°)	Implemented
7	Lighting — HemisphereLight (ambient sky/ground gradient)	Implemented
8	Lighting — DirectionalLight (fill/shadow light)	Implemented
9	Lighting — PointLight (orbiting inspection light)	Implemented
10	Shadow Mapping (PCFSoftShadowMap on PointLight)	Implemented
11	Wheel Auto-Rotation Animation (speed-proportional)	Implemented
12	Light Position Orbiting Around Truck (continuous animation)	Implemented
13	Keyboard — Drive Forward / Reverse (W / S or Arrow Up / Down)	Implemented

#	Feature	Status
14	Keyboard — Steer Left / Right (A / D or Arrow Left / Right)	Implemented
15	Keyboard — Brake (Space)	Implemented
16	Keyboard — Toggle Auto Camera Orbit (C)	Implemented
17	Keyboard — Toggle Rotating Inspection Light (L)	Implemented
18	Keyboard — Reset Scene to Initial State (R)	Implemented
19	Mouse — Drag to Orbit Camera (pointerdown + pointermove)	Implemented
20	Mouse — Scroll Wheel Zoom (wheel event, range 6–24 units)	Implemented
21	Smoke Particle System from Front Exhaust Pipe (H / Horn Button)	Implemented
22	Procedural Road Texture with Lane Markings	Implemented
23	Front Wheel Steering Pivot Animation (A / D keys)	Implemented
24	Scene Fog Effect (depth fog, near 18 – far 60)	Implemented

Table 02: Project Feature Table

5. Contribution

The following table summarizes the contribution of each group member:

Member	Student ID	Contribution
Ananto Nayan Bala	20210204028	• 3D truck modeling using Three.js • Custom vertex and fragment shader implementation • Procedural texture generation (truck body, wheels, road) • Lighting setup (Hemisphere, Directional, Point light) • Camera system with orbit, zoom, and auto-follow • Wheel rotation and vehicle movement animation
Tasmia Hossain	20210204038	• Keyboard-based vehicle control (drive, steer, brake) • Mouse interaction (camera rotation and zoom) • Smoke particle system implementation • Scene environment design (fog, background, layout)

Table 03: Member Contribution Table